

Hyperparameter Optimization:

- Hyperparameter are the parameters that defines model's architecture.
- Donot change with model training.
- Do not learn from model training, instead it controls learning process.
- Hyperparameters are set before training a ML model.

Each model has its own set of hyperparameters.

Examples:

- Linear / Logistic Regression: C, optimization algorithm, max_itr for gradient descent
- Decision trees: max depth, min leaf, criterion, min sample split
- Random Forest: no. of trees, max features, learning rate, subsample
- Neural network: learning rate, batch size, epochs, activation function, optimizer

Why Hyperparameter Tuning is important ?

- To Find optimal solution.
- Improve model performance. i.e accuracy, F1 score.
- Bad setting may leads to worse results even if data is good.
- Wrong hyperparameter can cause overfitting and under fitting.
- Can cause faster convergence. i.e learning rate.

- Reduces unnecessary and random fluctuations in prediction if hyperparameters are tuned well.
- Sensitive models like neural networks rely on hyperparameters.

Searching for hyperparameters.

1. GridSearch:

Gridsearch is a technique to tune hyperparameters. A predefined set of hyperparameters are given and it find the best combination for ML model.

There are 4 steps in gridsearch:

- Define the range of possible values for all hyperparameters.

```
param_grid = {  
    "C": [0.01, 0.1, 1, 10],  
    "solver": ["liblinear", "saga"]  
}
```

- Evaluate the model performance for each hyperparameter combination.
- Cross validate.

```
grid_search = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid,  
    scoring="roc_auc",  
    cv=5  
)
```

- Pick top performing hyperparameters.

Advantages:

- Finds optimal combinations.
- Any model with hyperparameters.
- Cross-validation reduces overfitting risk.

Disadvantages:

- Time consuming when it comes to large grids.
- more hyperparameters cause exponential growth in combinations.

2. Random Search:

tries n number of random combinations from a given hyperparameter space.

The hyperparameter values can be passed as list or as a continuous distribution.

doesn't matter how huge the parameters space is. We set the number of random combinations that need to be tried out.

There are 4 steps in random search:

- Define the random sampling space:
 - Explicit values
 - Or continuous distribution

```
param_dist = {  
    "C": uniform(0.01, 10),  
    "solver": ["liblinear", "saga"] }
```

- Sample hyperparameters randomly.
- Evaluate the model performance for each randomly sampled combination.

```
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=10,    combinations to try
    scoring="roc_auc",
    cv=5,
    random_state=42,
    verbose=1
)
```

- Cross validate
- Pick top performing hyper parameters

3. Bayesian optimization:

It is a probabilistic method for hyperparameter optimization.

Instead of trying all possible combination in sample space blindly it builds a probabilistic model of function mapping.

How it works ?

- hyper parameter sample is defined
- build a probabilistic model.
- Select the next hyperparameters to evaluate.
- Then evaluate selected hyperparameters
- Update probabilistic model.

- Repeat until convergence.

Advantages

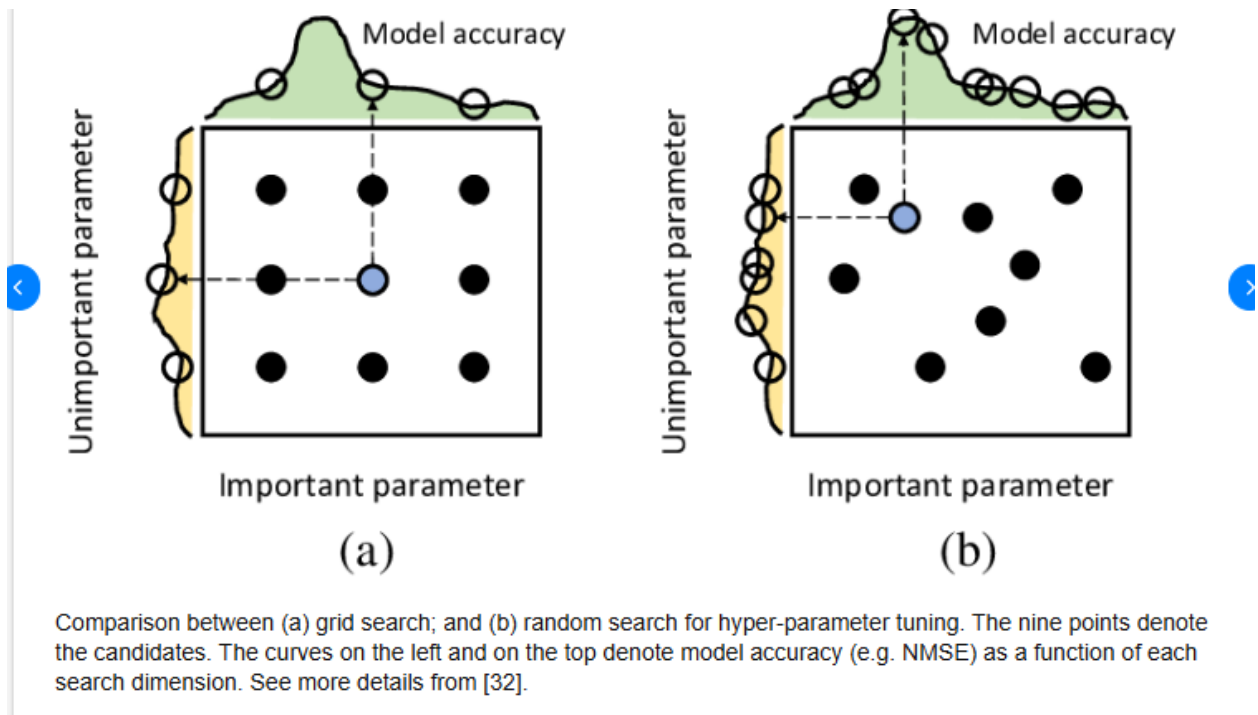
- High cost evaluations are done efficiently.
- Compared to grid search requires few evaluations
- Can find optimal hyperparameters faster than randomsearch

Disadvantages:

- More complex to implement.
- Choice of probabilistic model defines performance.
- Not ideal for very large very large models.

Why random search efficient than grid search ?

- Non sequential sampling: random search samples only random points not sequential.
- random search doesnot give equal importance to every hyperparameter combination.



Fine Tuning:

Process of making adjustments on a pretrained model to improve its performance.

We don't train from scratch, we just manipulates parameters.

It improve accuracy, robustness and generalization.

Fine tuning is done according to need of application.

Types of fine tuning:

i. **hyperparameter fine tuning:**

making adjustments in hyperparameter like learning rate, batch size, no. of epochs.

ii. Model parameter fine tuning:

Adjusting weight and biases. Usually done in deep learning.

iii. Layerwise fine tuning:

Some layers of pretrained model are freeze and train only last layers.

iv. Optimizer fine tuning:

Choosing right optimizers and their parameters.

We can also think types of fine tuning in much broader way:

1. Full parameter fine tuning:

Instead of adjusting only few parameters, it refers to updating all trainable parameters of a model during training.

Eg: chatgpt is likely to be full parameter fine tuned on top of its different gpt model.

2. Domain specific fine tuning:

Instead of training in large amount of data and heavy parameters combinations, it refers to adapting a pretrained model to a certain domain, industry.

Goal is to make model perform better in target domain.

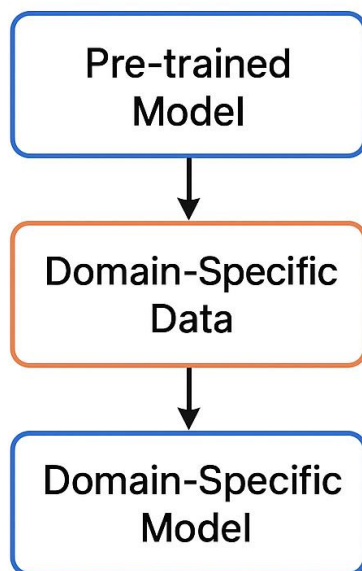
3. Specific task fine tuning:

It is the process of making a model capable to perform a specific task with high precision.

Goal is to make model excel in one task.

Eg: customer support chatbot.

Domain-Specific Fine-Tuning



Specific Task Fine-Tuning

